

Milestone 4

Model Training

Contents:

- Overview / Objective
- Dataset Details
- Architecture
- Initial Results
- Observations / Notes for the next Milestone
- Update on changes made
- Work division
- Summary of previous Milestones

Overview / Objective

- This fourth milestone focuses on the **implementation of the complete AI Stylist architecture**, as outlined in the previously demonstrated flowchart. It builds upon the work from earlier milestones to integrate all components — from data preprocessing to query handling and product recommendation — into a functional end-to-end system.
- This document provides:
 - Detailed information on the **dataset** used for catalog, customer, and transaction data.
 - A description of the **system architecture** and its workflow.
 - The **initial results** obtained after implementing the architecture.
 - **Observations** and notes for further improvement in the next milestone.
 - An **update on changes** made since the previous milestone.
 - The **division of work** among team members for this stage.

- And finally, a **summary of the previous milestones**, including data preprocessing and architecture design progress.
- **Link to Previous Milestones** [[Link](#)]

Dataset Details

- Description of datasets:
 - **Catalog** (before preprocessing):
 - File name: “articles.csv”
 - No. of features: 25
 - No. of samples: 105542
 - Feature description:
 - article_id
 - product_code
 - prod_name
 - product_type_no
 - product_type_name
 - product_group_name
 - graphical_appearance_no
 - graphical_appearance_name
 - colour_group_code
 - colour_group_name
 - perceived_colour_value_id
 - perceived_colour_value_name
 - perceived_colour_master_id
 - perceived_colour_master_name
 - department_no
 - department_name
 - index_code
 - index_name
 - index_group_no

- index_group_name
- section_no
- section_name
- garment_group_no
- garment_group_name
- detail_desc

○ **Catalog** (preprocessed):

- File name: “final_cloth_data_csv.csv”
- No. of features: 18
- No. of samples: 78,017
- Feature description:
 - **SKU_No:** Unique identifier assigned to each product.
 - **prod_name:** The product’s display or short name.
 - **product_type_name:** The specific type or model classification of the product.
 - **product_group_name:** The broader category under which the product falls.
 - **graphical_appearance_name:** The visual design or pattern category of the product.
 - **colour_group_name:** The primary color classification or group name.
 - **department_name:** The department or collection to which the product belongs.
 - **index_name:** Internal index categorizing the product within the department.
 - **index_group_name:** Higher-level grouping for similar index categories.
 - **section_name:** Store or catalog section (e.g., by gender, style, or use).

- **garment_group_name:** Group defining the garment's construction or functional type.
 - **detail_desc:** Detailed description of the product's material, design, or features.
 - **Eco_Score:** A numerical score representing the product's environmental sustainability.
 - **Style_Reward_Points:** Loyalty or style reward points earned upon purchase.
 - **Stock_Status:** Current availability of the product (e.g., "In Stock", "Low Stock").
 - **Price_INR:** Product's price in Indian Rupees (₹).
 - **Discount_Percentage:** Discount offered on the product (in percentage).
 - **Return_Type:** Return policy applicable to this product.
 - **Image_URL:** Path or URL of the product image for display.
- [Download link](#)

○ **Transactions data:**

- File name: "transactions_filtered_390k.csv"
- No. of features: 9
- No. of samples: 3,90,000
- Feature description:
 - **t_dat:** Transaction date representing when the purchase was made.
 - **article_id:** Unique identifier for the specific article or item sold.
 - **sales_channel_id:** Channel through which the sale occurred (e.g., 1 = Online, 2 = In-store).
 - **payment_method:** The payment method used for the transaction (e.g., Credit Card, UPI, Cash).
 - **SKU_No:** Unique identifier for the product (links to the product catalog).

- **prod_return_type:** Type of return policy associated with the product.
- **return_Status:** Indicates whether the item was returned or not.
- **delivered:** Delivery status of the product.
- **cust_id:** Unique identifier for the customer who made the purchase.
- [Download link](#)

○ **Customers data:**

- File name: “customers_matched_390k.csv”
- No. of features: 6
- No. of samples: 1,92,508
- Feature description:
 - **age:** Age range of the customer.
 - **postal_code:** The city or postal area associated with the customer’s address.
 - **gender:** Gender identity of the customer.
 - **size:** Preferred clothing size of the customer.
 - **cust_id:** Unique identifier assigned to each customer.
 - **embed_text:** Textual embedding or descriptive summary combining key customer attributes for use in text-based or embedding models.
- [Download link](#)

○ **Image Data:**

- Resolution: 1166×1750 Pixels
- No. of images: 77000
- [Download link](#)

Architecture

- The flow chart could be seen below the architecture details (at the end).
- **Application's flow is as explained:**
 - **Login and Log out:**
 - To login, the user must give a prompt in the ChatBot in this format “cust_id:{customer_id}”.
 - e.g. – “cust_id:1”, this will lead to logging in as customer 1.
 - To log out, the user needs to type “logout” in the ChatBot.
 - **CONTEXT** (Must be built before answering any user query. If context is missing, the system MUST ask for it — it cannot proceed.)
 - The context contains all the user details which are available in our database. This is maintained to get all the details about the customer in 1 place, so that there is no need to fetch the customer frequently. What is included in context:

1. User is Logged in

The system fetches data automatically from CSVs using pandas.

Context consists of three main parts :

- Customer Details (Columns: age, gender, size, cust_id)
- Transaction History (Columns: t_dat, SKU_No, payment_method, prod_return_type / return_Type, return_Status, delivered)
- Product Details (Columns: SKU_No, prod_name, product_type_name, product_group_name, colour_group_name, department_name, Price_INR, Discount_Percentage, Return_Type, Eco_Score)
- Valid customer IDs – 1,3,5.

2. User is logged out

The LLM must explicitly ask for age and gender before proceeding.

Once that information is provided, a minimal context is created using those values.

○ AI STYLIST

- The **AI Stylist** is responsible for understanding the user's fashion intent and recommending the most relevant clothing products. It personalizes results using the **customer's context** (age, gender, purchase history, etc.) and **semantic understanding** of the user query. Its done in the following 3 steps:

1. Query Refinement (refine_prompt)

The user query and customer context are passed to a dedicated LLM prompt called **refine_prompt**.

Its purpose is to **rewrite the user query** into a refined, detailed version that takes into account user-specific attributes.

Example:

```
Customer Context:
Gender: Female
Age: 25-30
Postal Code: Mumbai
--- PURCHASE TRANSACTIONS ---
Transaction 0: Purchased formal shirts and trousers
--- PRODUCTS PURCHASED ---
Product 0: Cotton Slim Shirt, Color: Blue
Product 1: Linen Regular Fit Pant, Color: Beige

User Query:
show me something to wear to a party

Refined Query:
show me trendy but elegant party outfits for a 25-year-old female from Mumbai, preferably in styles similar to her past purchases like shirts and pants, in blue
```

2. Searching Product Catalog (search_products)

The **refined query** and all product descriptions are converted into **numeric embeddings** using the **BGE model**.

Then the system:

1. Computes **cosine similarity** between the query embedding and every product embedding.
2. Measures how semantically close each product is to the query intent.
3. Sorts all products by similarity score.
4. Returns the **top 20 most relevant products**.

This ensures that results are driven by **semantic meaning**, not just keywords.

3. Ranking the Top 3 (recommend_top3_structured)

The top 20 products and the refined query are passed to another LLM called `recommend_top3_structured`, whose goal is to identify the top 3 most suitable products for the user.

Weightage Logic:

- 80% → Current user query meaning (highest priority).
- 15% → Customer context (e.g., gender, location, preferences).
- 5% → Purchase history (style consistency only).

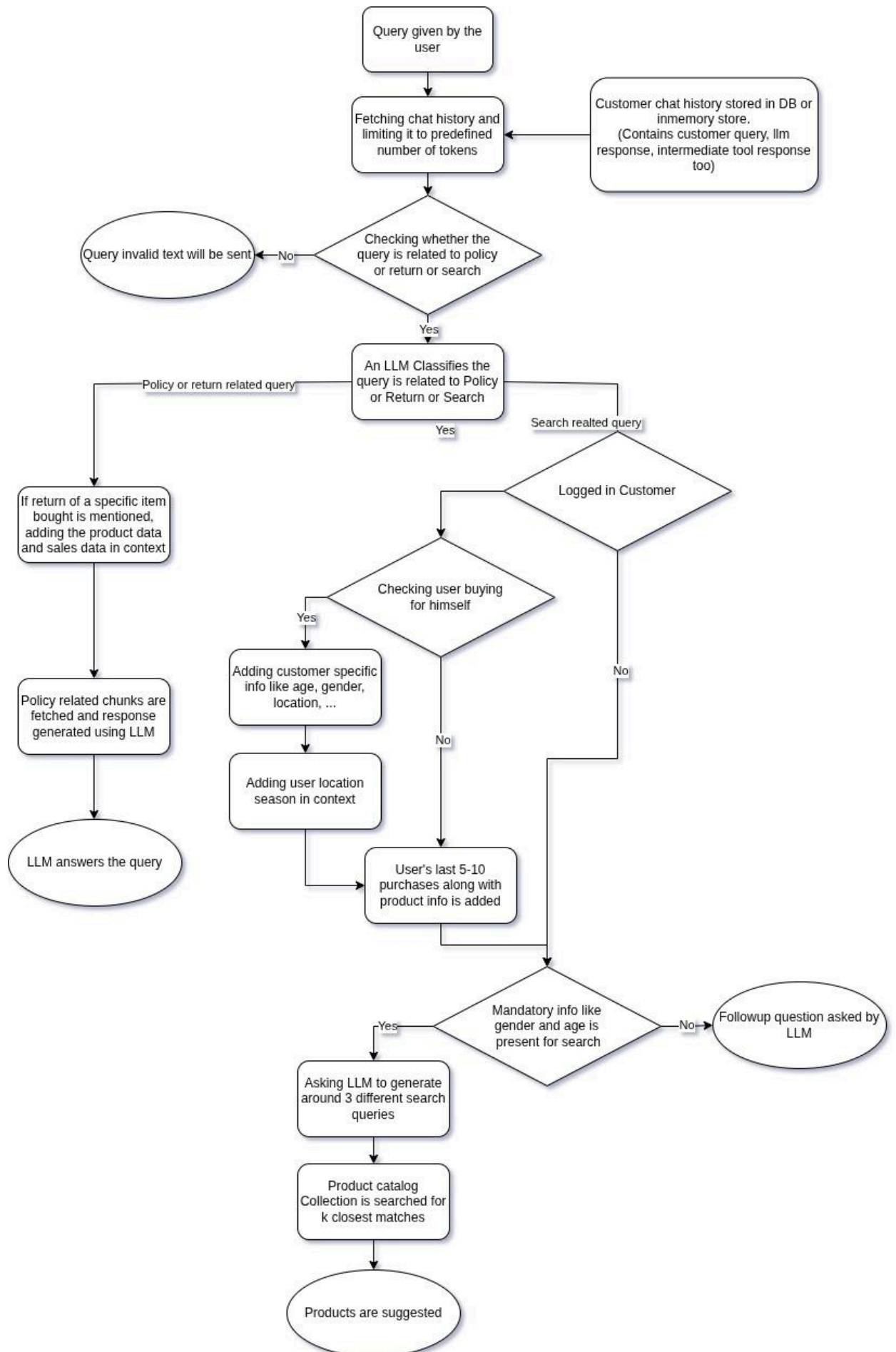
This prevents bias from past purchases — e.g., a user who bought party wear earlier but now asks for office wear will get relevant new options.

The LLM outputs the final top 3 products in a structured format.

- **POLICY LLM**

The **Policy LLM** handles all customer queries related to **store policies, returns, exchanges, and refunds**. It ensures accurate, personalized answers by combining stored policy data, and conversation history. It does the following things:

- The model identifies the **policy theme** of the user query (e.g., “return window,” “exchange,” “refund timeline”).
 - It retrieves the **relevant policy section** from stored data or rules.
 - The response is then generated in **natural, conversational form**, maintaining accuracy and empathy.
-
- **Flow chart: (accommodated in the next page)**



Initial Results

- When using minimal data (tested with minimal data first, because it was not always feasible to test on the whole data) while testing the results were not satisfactory.
- Without building context and without any followup questions the search results were generic and bland, and intended products were not fetched.
- When the user query spammed multiple queries the search results were losing its relevance. The model seemed to be struggling.

Update on Changes Made

- We focused on context building where without age and gender info, the product search will not be done.
- Also, we found that chat history needs to be maintained and fed into the llm for better understanding of customer intention and better search quality.

Work Division

- **Image data management - Neeraj and Harsha**
 - The data consists of both textual and image data, totalling to 30 GB. And, since it is not possible to put it on a G-Drive, Neeraj has downloaded the whole dataset from the website and kept it on his machine.
 - He has segregated 5 GBs of it for us to test it out while working on our individual parts, and after integrating it together, he is able to perform the whole pipeline on his machine locally.
- **Policy RAG and streamlit UI - Madhav and Manish**
 - Embedding policy doc,
 - Fetching chunks from policy doc
 - Prompt engineering for getting answer

- **Customer context building - Harsha and Manish**
 - Fetches customer meta from csv and build context like age, gender, location
 - If not a logged in customer then followup questions are asked
- **Product RAG - Harsha and Manish**
 - Classify query whether this is policy related or product search related - Manish
 - Product fetching from embedding
 - Product reranker chain
- **Product Embedding - Neeraj and Adarsh**
 - Embedding product meta and saving it in VDB
- **Memory and Customer session Management - Manish**
 - Memory and customer meta is maintained in a class instance
- **Preparing the submission document - Adarsh, Manish and Harsha**

Summary of previous Milestones

Milestone 1:

- This milestone consists of the problem definition and literature review. The project objective was divided into two parts, A and B. The first one being “Store policy + Customer help desk” and the other part is an “AI Stylist”.
- It then talks about the existing solutions, baselines and benchmarks. Taking a deep dive into them.
- Concluding that our suggestion system is going to use text features alone and will suggest clothes based on user inputs.
- The folder structure is as given below:
 - /group-5-DS-AI-Lab-Project
 - /milestones
 - /milestone-1
 - Milestone - 1.pdf

Milestone 2:

- This milestone had details about us choosing a dataset, and performing analysis on it.
- It had some sample images from the image dataset.
- The code for the EDA performed is also provided in a jupyter notebook.
- The FAQ and Policy details are provided in a pdf file with the same name.
- And the dataset for articles, customers detail and customers' purchase histories have been shared.
- The folder structure is as given below:
 - /group-5-DS-AI-Lab-Project
 - /milestones
 - /milestone-2
 - /sample_images
 - (a few images are here)
 - EDA_on_catalog.ipynb
 - Milestone - 2.pdf
 - articles.csv
 - customers.csv
 - faq_and_policy.pdf
 - purchase_history.xlsx

Milestone 3:

- The third milestone had intricate details about the RAG (Retrieval Augmented Generation) method and a handful of embedding models to compare their strengths and weaknesses.
- A jupyter notebook showing the hands-on code for the whole embedding pipeline.
- It starts the work with “filtered_data_with_images.csv” containing the textual details about the images, and embeds it using the “all-MiniLM-L6-v2” model.
- These vectors and the meta data are then saved for later use.
- And, then using the FAISS (Facebook AI Similarity Search) to deal with the search queries.
- Finally concluding with some sample queries and their results.

- The folder structure is as given below:
 - /group-5-DS-AI-Lab-Project
 - /milestones
 - /milestone-3
 - EmbeddingTest.ipynb
 - Milestone - 3.pdf